

ПРАКТИЧЕСКАЯ РАБОТА 7

OpenCV на Python

Компьютерное зрение — это перспективное направление развития технологий, позволяющее обучить компьютер навыкам распознавания изображений и видео. С помощью компьютерного зрения компьютеры могут не только анализировать и понимать визуальную информацию, такую как изображения и видео, но и принимать решения на основе увиденного. Так автопилот, управляющий автомобилем, может анализировать изображения, поступающие с камер и принимать решения на основании данной информации. Компьютерное зрение на производстве позволяет выявлять износ различных деталей до того, как это приведет к поломке.

OpenCV поддерживает платформы глубокого обучения для обработки изображений и видео. В компьютерном зрении основным элементом является выделение пикселей из изображения, чтобы изучить объекты и, таким образом, понять, что они содержат.

Вот несколько ключевых аспектов, которые компьютерное зрение стремится распознать на фотографиях: обнаружение объектов, распознавание объектов, различные виды классификации, сегментация, то есть выделение пикселей, принадлежащих объекту, и многое другое.

Алгоритм водораздела

Мы можем анализировать изображения с помощью различных алгоритмов. Рассмотрим принцип работы алгоритма водораздела (Watershed) на примере топографии. Пусть у нас есть картинка в оттенках одного цвета, рассмотрим ее как топографическую поверхность, где высокая интенсивность обозначает вершины и холмы, а низкая — долины. Мы начинаем заполнять все изолированные долины (локальные минимумы) водой разного цвета (метками). По мере подъема уровня воды, в зависимости от близлежащих вершин (градиентов), вода из разных долин, очевидно, разного цвета, начнет сливаться. Чтобы избежать этого, мы строим барьеры в местах слияния вод. Мы продолжаем заполнять водой и возводить барьеры до тех пор, пока все вершины не окажутся под водой. Затем созданные вами барьеры дают результат сегментации. В этом и заключается принцип работы алгоритма Watershed.

Нашей основной проблемой при таком подходе является чрезмерная сегментация из-за шума или любых других неровностей в топографии нашего изображения. В OpenCV реализован алгоритм разделения на основе маркеров, в котором вы указываете, какие точки закрашиваемой долины должны быть объединены, а какие — нет. Все, что мы делаем, — это даем разные обозначения нашему знакомому объекту. Обозначьте область, в которой мы уверены, что это передний план или объект, одним цветом (или интенсивностью), мы обозначаем область, в которой уверены, что это фон или не объект, другим цветом и, наконец, область, в которой мы ни в чем не уверены, обозначаем ее 0. Это наш маркер. Затем примените алгоритм водораздела. Тогда наш маркер будет обновлен с помощью меток, которые мы указали, и границы объектов будут иметь значение -1.

Пример с монетами

Пример того, как использовать преобразование расстояния (Distance Transform) вместе с водоразделом для сегментации взаимно соприкасающихся объектов.



На изображении монеты соприкасаются друг с другом, то есть посчитать их количество программным путем сейчас будет довольно проблематично. Даже если вы установите пороговое значение, они будут соприкасаться друг с другом. Мы начнем с определения приблизительной стоимости всех монет. Для этого мы можем использовать бинаризацию Otsu. Это алгоритм вычисления порога бинаризации для полутонового изображения, используемый в области компьютерного распознавания образов и обработки изображений для получения черно-белых изображений. Назван в честь доктора инженерии Токийского университета Нобуюки Отсу.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('coins.png')
assert img is not None, "file could not be read, check with os.path.exists()"
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
ret, thresh = cv.threshold(gray,0,255,cv.THRESH_BINARY_INV+cv.THRESH_OTSU)
```

Результатом работы данного алгоритма будет следующее:



Теперь нам нужно удалить все мелкие белые шумы на изображении. Мы точно знаем, что область, расположенная ближе к центру объектов, находится на переднем плане, а область, удаленная от объекта, — на заднем. Единственная область, в которой мы не уверены, — это область границ монет.

Соответственно, нам нужно выделить область, в которой, как мы полагаем, находятся монеты. Поскольку они соприкасаются друг с другом, еще одним хорошим вариантом было бы найти преобразование расстояния и применить соответствующий порог. После этого нам нужно будет найти область, в которой, как мы полагаем, не находятся монеты. Для этого мы увеличиваем границу объекта по отношению к фону. Таким образом, мы можем убедиться, что любая область на заднем плане действительно является фоном, поскольку граничная область удалена. Посмотрим изображение ниже.



Остальные области — это те, о которых мы не имеем ни малейшего представления, будь то монеты или фон, и алгоритм поиска должен помочь нам найти их. Обычно эти области находятся вокруг грани монет, где пересекаются передний план и фон (или даже две разные монеты). Мы называем это границей. Его можно получить, вычитая область sure_fg из области sure_bg.

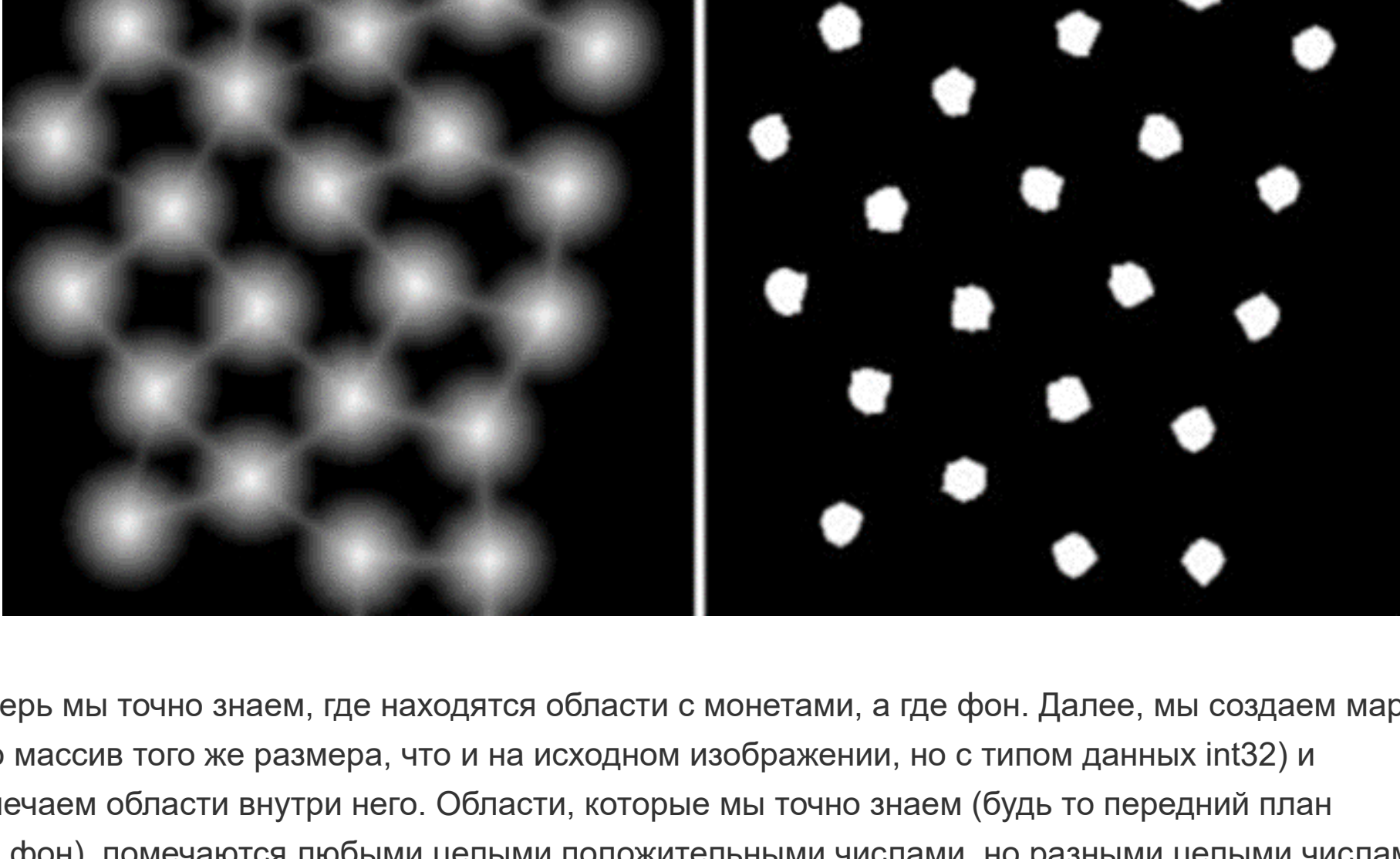
```
# удаляем шумы
kernel = np.ones((3,3),np.uint8)
opening = cv.morphologyEx(thresh,cv.MORPH_OPEN,kernel, iterations = 2)

# область фона, в которой мы уверены
sure_bg = cv.dilate(opening,kernel,iterations=3)

# область переднего плана, в которой мы уверены
dist_transform = cv.distanceTransform(opening,cv.DIST_L2,5)
ret, sure_fg = cv.threshold(dist_transform,0.7*dist_transform.max(),255,0)

# неизвестная область
sure_fg = np.uint8(sure_fg)
unknown = cv.subtract(sure_bg,sure_fg)
```

Посмотрим, что получилось в итоге. На изображениях ниже с пороговым значением (справа) мы помечаем несколько областей монет, что это монеты, и теперь они отделены. В некоторых случаях нас может интересовать только сегментация переднего плана, а не разделение соприкасающихся объектов, и тогда мы можем использовать вместо преобразования расстояния, простое размытие изображения (слева).



Теперь мы точно знаем, где находятся области с монетами, а где фон. Далее, мы создаем маркер (это массив того же размера, что и на исходном изображении, но с типом данных int32) и помечаем области внутри него. Области, которые мы точно знаем (будь то передний план или фон), помечаются любыми целыми положительными числами, но разными целыми числами, а область, которую мы точно не знаем, просто оставляется равной нулю. Для этого мы используем cv.connectedComponents(). Он помечает фон изображения значением 0, затем другие объекты помечаются целыми числами, начинающимися с 1.

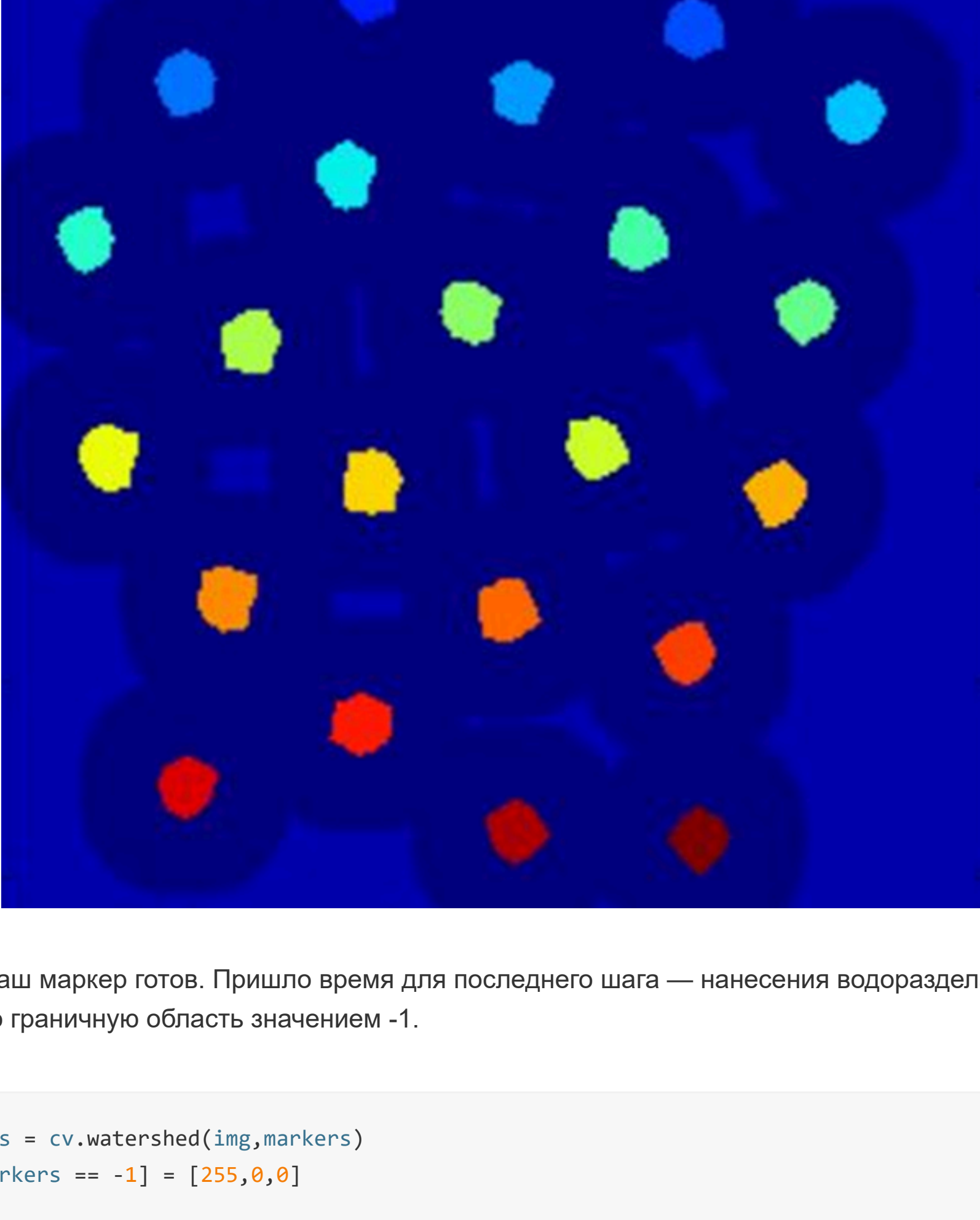
Но мы знаем, что, если фон помечен 0, алгоритм Watershed будет рассматривать его как неизвестную область. Поэтому мы хотим отметить его другим целым числом. А нулем мы будем помечать не фон, а действительно неизвестную область.

```
# помечаем маркер
ret, markers = cv.connectedComponents(sure_fg)

# фон это теперь не 0 а 1
markers = markers+1

# неизвестная область этот 0
markers[unknown==255] = 0
```

Посмотрим результат на цветовой карте JET. Темно-синяя область показывает неизвестную область. Монеты Sure окрашены в разные цвета. Остальные области, которые мы определили, показаны более светлым синим цветом по сравнению с неизвестной областью.



Теперь наш маркер готов. Пришло время для последнего шага — нанесения водораздела, для этого граничную область значением -1.

```
markers = cv.watershed(img,markers)
img[markers == -1] = [255,0,0]
```

Посмотрим, что получилось в итоге.



Как видно, границы у большинства монет определились достаточно точно, хотя кое-где граница все же проходит не совсем корректно.